Exp No 6 Error Detection and Correction
Date: 8/8/24 using Hamming code

AIM:-

To write a program to implement error detection and correction using the Hamming code

ALGORITHM:-

1. Start the program
2. Read the number of 4 bit data blocks and the input data bits.
3. Arrange the data and parity bits in a 7-bit hamming code.
4. Calculate the parity bits using even parity and generate the hamming code.
5. Introduce a single bit error at the specified position.
6. At the receiver calculate the syndrome to identify the error position.
7. Correct the error by inverting the faulty bit.
8. Extract and display the recovered original data bits.

Program:-

```
def encode(d):
    h = [0]*7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h.
def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0
        print (f"Error detected at bit position: {error_pos}
                -> correcting... ")
        h[error_pos - 1] ^= 1
    else:
        print ("No error detected.")
    return error_pos.
def to_str(bits)
    return "".join(str(b) for b in bits)
def main():
    Print ("=== Hamming's code - Error Detection
            And correction ===\n")
    n = int(input("Enter. number of.. 4 bit
        data blocks in the stream: "))
```

```
for b in range(1, n+1):
    print(f"\n--- Block {b} ---")
    d = list(map(int, input("Enter 4 data bits
        (space separated, e.g. 1 0 1 1): ").split()))
    h = encode(d)
    print(f"Original Data Bits : {to_str(d)}")
    print(f"Encoded Hamming code (7 bits):
                {to_str(h)}")

    pos = int(input("Enter bit position to introduce
        error (1-7, 0 = no error): "))
    if 1 <= pos <= 7:
        h[pos-1] ^= 1
        print(f"Transmitted code (with error
        induced at pos {pos}): {to_str(h)}")
    else:
        print(f"Transmitted code (no error
            induced): {to_str(h)}")

    print(f"Received code: {to_str(h)}")
    print("Checking for errors...")
    detect_and_correct(h)
    print(f"Corrected code : {to_str(h)}")
    rd = [h[2], h[4], h[5], h[6]]
    print(f"Recovered Data bits: {to_str(rd)}")
if __name__ == "__main__":
    main()
```

Output :-

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===
Enter number of 4-bit data blocks in the stream: 2
--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1011): 1011
Original Data Bits: 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0=no error): 3
Transmitted Code (with error induced at pos 3): 0100011
Received Code : 0100011
Checking for errors...
    Error detected at bit position: 3 → Correcting...
Corrected Code : 0110011
Recovered Data Bits : 1011
--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1011): 0110
Original Data Bits : 0110
Encoding Hamming Code (7 bits): 1100110
Enter bit position to introduce error (1-7, 0=no error): 6
Transmitted Code (with error induced at pos 6): 1100100
Received Code : 1100100
Checking for errors...
    Error detected at bit position: 6 → Correcting...
Corrected Code : 1100110
Recovered Data Bits : 0110
```

Result :-

The program to implement error detection and correction using Hamming code concept was successfully written in Python and executed, verified using a sample input data stream.